Evaluation Report: Automating End-to-End Functional Testing of Web Applications based on Augmented Workflows

Report Date: 2026-03-12
Repository: `softscanner_fa`
Branch: `feat/phase-b`
Evaluation Scope: Phases A1, A2, B0, B1, B2 (generation stages complete); B3/B4 (execution and coverage stages pending)
Subjects: 6 Angular applications
Evaluator: Automated pipeline + GT-driven adjudication

1. Executive Summary

This report documents the evaluation of a static-analysis-based approach for automated end-to-end (E2E) functional test generation for Angular web applications. The approach constructs a finite, constraint-aware workflow reference space *W* directly from frontend source code and then generates executable Selenium WebDriver tests for every workflow in *W*.

Evaluation Scope

The evaluation covers the complete pipeline from static extraction through deterministic test code generation across six Angular applications of varying size and architectural complexity. Phase B execution (running the generated tests against live applications) is not yet completed and is documented as future work.

Evaluation Subjects

Six Angular applications were used:

Subject	Framework	Workflows	Generation
posts-users-ui-ng	Angular 18	18	18/18
spring-petclinic-angular	Angular 14	74	74/74
heroes-angular	Angular 14 (NgRx)	19	19/19
softscanner-cqa-frontend	Angular 17.3	16	16/16
ever-traduora	Angular 12.2	109	109/109
airbus-inventory	Angular 12.2	21	21/21
Total	Angular 12–18	**257**	**257/257**

Artifacts Evaluated

All pipeline stages through B2 have been evaluated:

- **A1:** Six UI interaction multigraphs (665 nodes, 1345 edges total)
- **A2:** 257 task workflows enumerated and classified (145 FEASIBLE, 112 CONDITIONAL, 0 PRUNED)
- **B0:** Six subject manifests validated (6/6 VALID, 0 errors)
- **B1:** 257 RealizationIntents derived; 257 ActionPlans generated and GT-validated (257/257 exact match)
- **B2:** 257 Selenium WebDriver TypeScript test files generated (100% generation rate)

Key Findings

1. **Generation coverage is 100%**: The pipeline successfully derives a plan and generates a syntactically valid Selenium test for every workflow in the reference space *W* across all six subjects.
2. **GT validation is exact**: B1 outputs (intents and plans) match all 257 ground-truth entries with zero mismatches, providing high confidence in the correctness of the planning stage.
3. **The pipeline is deterministic**: All stages (A1, A2, B0, B1, B2) are byte-identical across independent runs, enabling reproducible evaluation.
4. **Constraint modeling is comprehensive**: The approach correctly identifies and handles guard-protected workflows (94 auth-required plans across 2 subjects), parameterized routes (34 param-dependent plans), and form-submission workflows (30 WSF-triggered plans).
5. **Execution coverage remains an open question**: B3 test execution has not been performed at scale. Generation coverage (C1+C2 = 100%) is a prerequisite for, but not a proof of, execution coverage (C3).

Comparison to AutoE2E

A companion evaluation of AutoE2E (Allan et al., ICSE 2025) on the same six subjects documents that AutoE2E does not generate executable test files — it produces a state graph and feature database from runtime exploration.

2. Evaluation Objectives

2.1 Research Questions

The evaluation is structured around three research questions (RQs):

RQ1 (Representation): *How can frontend interaction behavior be represented such that the resulting workflow space is complete with respect to the implementation, constraint-aware, finite, and executable?*

RQ2 (Construction): *How can such a representation be systematically and deterministically derived from frontend source code?*

RQ3 (Realization): *How can workflows in this representation be instantiated and executed so as to maximize coverage over that space?*

2.2 What Each Pipeline Stage Must Demonstrate

Stage	RQ Addressed	What Must Be Demonstrated
A1	RQ1, RQ2	That the multigraph faithfully encodes the application's interaction structure with auditable evidence; that extraction is deterministic
A2	RQ1, RQ2	That the workflow enumeration algorithm produces a finite, classified workflow space *W* that is complete w.r.t. the A1 multigraph
B0	RQ3	That per-subject runtime configuration is expressible and validatable without requiring live execution
B1	RQ3	That deterministic planning maps every workflow to a concrete, assignment-bound browser action sequence
B2	RQ3	That deterministic code generation produces syntactically valid, framework-correct test code for every planned workflow
B3	RQ3	That generated tests execute and pass on running applications (pending)
B4	RQ3	That coverage metrics can be computed faithfully over the fixed space *W* (pending)

2.3 Coverage Metric Definitions

Generation Coverage (C1 + C2): Measures whether the pipeline can plan and generate code for every workflow in *W*.

- *C1 (Plan Coverage)*: `|plans produced| / |W|` — fraction of workflows that produce a valid ActionPlan. A workflow is counted if `b1-plans.json` includes it with `skipped = false`.

- *C2 (Code Coverage)*: `|tests generated| / |W|` — fraction of workflows for which B2 produces a syntactically valid `.test.ts` file. Currently co-extensive with C1.

Both C1 and C2 are fully evaluated and equal 257/257 = 100% across all subjects.

Execution Coverage (C3) — Future Stage: `|E| / (|W| - |W_app_not_ready|)` where *E* is the set of workflows for which at least one B3 test run passes, and workflows failing solely due to `FAIL_APP_NOT_READY` are excluded from the denominator. C3 cannot be evaluated until B3 is implemented.

Validity Coverage (C4) — Future Stage: Measures the richness of postcondition assertions beyond URL-match and no-crash. Currently deferred; all generated tests include `assert-url-matches` and `assert-no-crash`, but deeper semantic assertion is not yet implemented.

3. Evaluation Corpus

3.1 Subject Selection Rationale

Six Angular applications were selected to cover the following dimensions of variation:

- Angular versions from 12 to 18
- Presence or absence of route authentication guards
- Application scale (small 16-workflow to large 109-workflow)
- Backend dependency types (JSON Server, Spring Boot, Express, none)
- Module architecture (flat single-module to highly modular)
- UI patterns (standard forms, Angular Material, NgRx, dialog composition)

3.2 Subject Descriptions

Subject 1 — posts-users-ui-ng

- *Framework:* Angular 18 + Express.js (in-memory backend)
- *Purpose:* Social platform for creating and browsing posts and users
- *Architecture:* 2 modules, 8 routes, 11 components, 41 widgets, 2 services. Flat module structure with standard reactive forms and routerLink navigation.
- *Auth:* None
- *Subject manifest:* `subjects/posts-users-ui-ng/subject-manifest.json`
- *Inclusion rationale:* Small, well-structured public application; tests basic navigation, form submission, and parameterized route workflows

Subject 2 — spring-petclinic-angular

- *Framework:* Angular 14 + Spring PetClinic REST API (Docker)
- *Purpose:* Veterinary clinic management — owners, pets, vets, specialties
- *Architecture:* 16 modules, 24 routes, 22 components, 120 widgets, 9 services. Highly modular with route hierarchy (ROUTE_HAS_CHILD), parameterized routes, and CRUD patterns.
- *Auth:* None (backend access without authentication)
- *Subject manifest:* `subjects/spring-petclinic-angular/subject-manifest.json`
- *Inclusion rationale:* Medium-sized application with the richest workflow space; tests complex multi-step plans, route parameter binding, and form-centric workflows. One of the AutoE2E benchmark subjects.

Subject 3 — heroes-angular

- *Framework:* Angular 14 (NgRx/Data) + JSON Server backend
- *Purpose:* Character management application (heroes and villains)
- *Architecture:* 5 modules (lazy-loaded features), 5 routes, 17 components, 28 widgets, 2 services, 10 external link targets. Uses NgRx/Data with entity services; lazy-loaded module routing with deduplication requirements.
- *Auth:* None
- *Subject manifest:* `subjects/heroes-angular/subject-manifest.json`

- *Inclusion rationale:* Tests lazy-loaded route deduplication, external URL navigation (11 WNE workflows), and NgRx state management patterns. One of the AutoE2E benchmark subjects.

Subject 4 — softscanner-cqa-frontend

- *Framework:* Angular 17.3 standalone components + Express.js backend
- *Purpose:* Software quality assessment platform based on ISO 25010
- *Architecture:* 3 modules, 1 route, 10 components, 21 widgets, 2 services. Single-route SPA with Angular Material components; dialog/modal composition (14 COMPONENT_COMPOSES_COMPONENT edges).
- *Auth:* None
- *Subject manifest:* `subjects/softscanner-cqa-frontend/subject-manifest.json`
- *Inclusion rationale:* Tests single-route SPA patterns, Angular 17 standalone components, and dialog precondition derivation

Subject 5 — ever-traduora

- *Framework:* Angular 12.2 + NestJS + MySQL (Docker Compose)
- *Purpose:* Translation management platform
- *Architecture:* 4 modules, 20 routes, 45 components, 152 widgets, 26 services. Largest graph (247 nodes, 499 edges). 63 CONDITIONAL workflows from auth guards, expression-gated UI, and parameterized routes.
- *Auth:* Yes — route guards (`AuthGuard` , `CanGuard` , `NoAuthGuard`)
- *Subject manifest:* `subjects/ever-traduora/subject-manifest.json`
- *Inclusion rationale:* Tests authentication workflow derivation, guard-based constraint modeling, and deep module hierarchy. One of the AutoE2E benchmark subjects.

Subject 6 — airbus-inventory

- *Framework:* Angular 12.2 + Spring Boot 2.5.5 + MySQL 5.7
- *Purpose:* Inventory management system for product CRUD operations
- *Architecture:* 1 module (flat), 7 routes, 10 components, 36 widgets, 5 services. All product routes protected by `CanActivateRouteGuard` (JWT token check).
- *Auth:* Yes — JWT-based route guard on all product routes
- *Subject manifest:* `subjects/airbus-inventory/subject-manifest.json`
- *Inclusion rationale:* Tests JWT authentication pattern, flat module architecture, and comprehensive CRUD service call modeling (19 COMPONENT_CALLS_SERVICE edges)

3.3 Corpus Summary Table

#	Subject	Angular	Nodes	Edges	Routes	Components	Widgets	Auth	Manifested
1	posts-users-ui-ng	18	64	131	8	11	41	No	`subjects/posts-users-ui-ng/`
2	spring-petclinic-angular	14	191	422	24	22	120	No	`subjects/spring-petclinic-angular/`
3	heroes-angular	14 NgRx	67	88	5	17	28	No	`subjects/heroes-angular/`
4	softscanner-cqa-frontend	17.3	37	74	1	10	21	No	`subjects/softscanner-cqa-frontend/`
5	ever-traduora	12.2	247	499	20	45	152	Yes	`subjects/ever-traduora/`
6	airbus-inventory	12.2	59	131	7	10	36	Yes	`subjects/airbus-inventory/`
	Total	12–18	**665**	**1345**	**65**	**115**	**398**		

4. Ground Truth Construction

4.1 Definition

Ground truth (GT) for this evaluation represents the expected output of the B1 planning stage for each workflow in the A2 workflow space. A GT entry specifies:

- The workflow ID (= A2 trigger edge ID)
- The expected start route (navigated before the test action)
- The expected assignment (account credentials if auth is needed; route param values; form data)
- The expected precondition sequence (types and configurations)
- The expected action step count and types
- The expected postcondition assertions

GT entries do not specify exact locator values or element attribute text, which are derived deterministically from A1 widget data. GT validates planning decisions, not selector specifics.

4.2 GT Construction Process

Ground truth was constructed manually in two passes:

Pass 1 — Initial construction (2026-03-10): A human expert enumerated 257 expected plans by reading the A2 workflow set for each subject alongside the application source code. GT was written as structured JSON entries in `docs/analysis/phase-b/gt/<subject>.json`. Each entry encodes the human's expectation for how a given workflow should be realized.

Pass 2 — Adjudication (2026-03-12): After B1 implementation was complete, the B1 runner compared each plan against its GT entry and reported mismatches. 150 initial mismatches were found across all subjects. Each mismatch was investigated and resolved into one of four categories:

Category	Count	Resolution
GT over-specification (human error)	~100	GT entry repaired per spec rules
Normative gap (spec silent on edge case)	5	Normative amendment frozen in `approach.md`
Validator comparison error	3	Comparison logic fixed
Implementation bug	2	Plan derivation code fixed

4.3 GT Repair Summary by Subject

Subject	GT Entries	Repairs Applied	Categories
posts-users-ui-ng	18	7	navigateUrl, stepCount
heroes-angular	19	4	navigateUrl (start route)
softscanner-cqa-frontend	16	2	dialog precondition removal
spring-petclinic-angular	74	116	navigateUrl, routeParams, formDataKeys, terminalUrl
ever-traduora	109	123	routeParams, account parity, navigateUrl, dialog preconditions
airbus-inventory	21	16	account, routeParams, navigateUrl, authSetup, terminalUrl
Total	**257**	**266**	

The high repair count for spring-petclinic-angular and ever-traduora reflects the complexity of those subjects (74 and 109 workflows respectively) and the systematic nature of GT over-specification — for example, the human initially over-constrained `terminalUrl` values for parameterized routes across dozens of entries.

4.4 Normative Amendments Frozen During GT Adjudication

Five decisions that were ambiguous in the original spec were clarified and frozen in `docs/paper/approach.md` during adjudication:

Decision	Rule Frozen
Start Route Selection	Exclude wildcards → prefer unguarded → fewest params → shortest path → alphabetical
Route Param Scope	`assignment.routeParams` includes params from both the selected start route AND the terminal route path
Form Field Scope	All A1-extracted form fields are included in the form schema without visibility or editability filtering
Auth Materialization	Auth precondition is emitted based on the selected start route's guards only (not all `startRouteIds`)
Dialog Precondition	A component is a dialog candidate iff it is a COMPONENT_COMPOSES_COMPONENT target AND its selector matches `/dialog

4.5 Role of GT in Evaluation

GT provides two guarantees:

1. **Planning correctness:** The B1 planner produces exactly the expected output for every workflow in the space, as defined by human expert review. This gives high confidence that the planning logic matches human intent.
2. **Regression detection:** Any future change to B1 or the underlying spec that alters planning output will be immediately detected as a GT mismatch. This makes the evaluation protocol self-enforcing.

GT does **not** address:

- Whether generated locators will succeed at test execution time (a B3 concern)
- Whether the postcondition URL assertions are semantically correct
- Whether entity data will be available at execution time

5. Evaluation Pipeline

5.1 Pipeline Overview

```
Frontend Source Code (Angular project)
         │
         ▼
    ┌─────────┐
    │   A1    │  Static extraction: Angular AST → UI Interaction Multigraph
    │ Extract │  Output: a1-multigraph.json
    └─────────┘
         │ a1-multigraph.json
         ▼
    ┌─────────┐
    │   A2    │  Graph algorithms: Multigraph → Workflow Space W
    │ Enumerate│  Output: a2-workflows.json
    └─────────┘
         │ a2-workflows.json
         ▼
    ┌─────────┐
    │   B0    │  Manifest authoring + validation
    │ Validate│  Input: subject-manifest.json (manual)
    └─────────┘
         │ subject-manifest.json + a2-workflows.json + a1-multigraph.json
         ▼
    ┌─────────┐
    │   B1    │  Deterministic planning: W × Manifest → ActionPlans
    │  Plan   │  Output: b1-intents.json, b1-plans.json
    └─────────┘
         │ b1-plans.json
         ▼
    ┌─────────┐
    │   B2    │  Deterministic code gen: ActionPlans → Selenium tests
    │  Gen    │  Output: tests/*.test.ts, b2-tests.json
    └─────────┘
         │
    [B3/B4 pending: execution, coverage reporting]
```

5.2 Stage-by-Stage Description

Stage A1: UI Interaction Multigraph Extraction

Input: Angular project root directory, TypeScript configuration file.

Output: `a1-multigraph.json` — a deterministic, auditable JSON representation of the application's UI interaction structure.

Process: A1 performs static analysis of the Angular project using `ts-morph` for TypeScript AST access and `@angular/compiler` for template analysis. It constructs a unified multigraph with:

- 6 node kinds: Module, Route, Component, Widget, Service, External
- 18 edge kinds: 11 structural (existence, ownership, containment) + 7 executable (interactions)
- Every node and edge backed by `SourceRef` (file + character offsets) for auditability
- `ConstraintSurface` on every executable edge capturing guards, route parameter requirements, and UI-state atoms

Validation performed: The A1 output is validated structurally (node/edge counts, sort order, integrity invariants) and compared against expected counts from `docs/validation/subjects.md`. Determinism is verified by running the extractor twice on each subject and comparing the SHA-256 hash of the JSON output.

Deterministic guarantee: Same codebase + same tsconfig → byte-identical `a1-multigraph.json`. Guaranteed by stable node/edge ID derivation, deterministic sort order (nodes by id, edges by from/kind/to/id), and deterministic AST traversal.

Stage A2: Task Workflow Enumeration and Classification

Input: `a1-multigraph.json`

Output: `a2-workflows.json` — the workflow space *W* as classified TaskWorkflows.

Process: A2 runs as pure graph algorithms over the frozen A1 multigraph. For each component-bearing route, it computes the active widget set and enumerates one TaskWorkflow per enabled trigger edge (kinds: WTH, WSF, WNR, WNE). Each workflow includes:

- Handler-scoped effect closure (CCS + CNR edges via `effectGroupId`)
- Deterministic redirect closure (cycle-safe, visit-cap protected)
- Merged constraint surface: `C(w)` = set-union of all step constraints
- Feasibility verdict (FEASIBLE / CONDITIONAL / PRUNED) by strict rule-based classifier

Validation performed: Workflow counts compared against expected values per subject. Classification breakdown (F/C/P) verified against ground-truth expectations. Determinism verified by two independent runs per subject.

Deterministic guarantee: Same `a1-multigraph.json` → byte-identical `a2-workflows.json`. Guaranteed by deterministic route iteration order, deterministic widget sort, deterministic effect collection by `callsiteOrdinal`, and deterministic redirect closure selection.

Stage B0: Subject Manifest Validation

Input: `subject-manifest.json` (human-authored), `a2-workflows.json`

Output: Validation log (`logs/b0-summary.json`)

Process: B0 validates each subject manifest against two checks:

1. *Schema validation*: all required fields present, correct types, optional fields (executionConfig, authSetup, formDataOverrides, skipWorkflows) conform to schema if present
2. *A2 cross-check*: every required guard name has at least one satisfying account; every required route param has a binding; every `skipWorkflows` and `formDataOverrides` key references an existing workflow ID

Validation performed: All 6 manifests validated (6/6 VALID, 0 errors, 0 warnings).

Stage B1: RealizationIntent Derivation + ActionPlan Generation

Input: `a1-multigraph.json`, `a2-workflows.json`, `subject-manifest.json`

Output: `b1-intents.json` (RealizationIntents), `b1-plans.json` (ActionPlans)

B1.1 — Intent derivation: For each TaskWorkflow, B1 reads A1 widget data and A2 workflow metadata to derive a `RealizationIntent` that captures: trigger widget attributes, start route URL template and required params, form schema (for WSF triggers), guard names, and effect step list. This derivation is deterministic and does not require the manifest.

B1.2 — Plan derivation: For each intent, B1 reads the manifest to bind concrete values and produces an `ActionPlan` with:

- `Assignment`: selected account (from manifest), bound route params, form data values
- `PreConditions`: `auth-setup` (if start route requires auth), `navigate-to-route` (always), `trigger-dialog-open` (if trigger is inside a dialog component)
- `ActionSteps`: one step per effect edge (click, type, clear-and-type, submit, select-option, wait-for-element, wait-for-navigation, wait-for-dialog)
- `PostConditions`: `assert-url-matches` (URL template with bound params) + `assert-no-crash`

Each step includes a `ScopedLocator` with one of 10 strategies (`id`, `name`, `formcontrolname`, `aria-label`, `routerlink`, `href`, `placeholder`, `data-testid`, `tag-position`, `custom`) plus optional component/form ancestor scoping.

Validation performed: Every ActionPlan compared against its GT entry (257/257 match). Determinism verified by two independent runs.

Stage B2: Selenium Test Code Generation

Input: `b1-plans.json`, `subject-manifest.json` (for `baseUrl`)

Output: `output/<subject>/tests/*.test.ts`, `output/<subject>/json/b2-tests.json`

Process: B2 emits one TypeScript file per ActionPlan. Each file is a standalone Selenium WebDriver test that:

1. Creates a Chrome headless WebDriver instance with implicit wait (10s) and navigation wait (15s)
2. Executes each precondition in order (auth: navigate to login → fill credentials → submit; navigate-to-route: `driver.get(BASE_URL + path)`; dialog-open: find and click the dialog opener)
3. Executes each action step using `driver.findElement(by).click()` / `.sendKeys()` / `.submit()` with optional ancestor scoping via nested `findElement` chains
4. Asserts each postcondition (URL contains check with `driver.wait` for navigation; no-crash via test reaching `console.log('Test PASSED:...')`)
5. Calls `driver.quit()` in `finally` block; exits with code 1 on failure

B2 translates `ScopedLocator` to Selenium `By.*` expressions: `id` → `By.id()`, `name` → `By.name()`, `formcontrolname` / `aria-label` / `routerlink` / `href` / `placeholder` / `data-testid` → `By.css('[attr="value"]')`, `tag-position` → `By.css('tag:nth-of-type(n)')`, `custom` → `By.css(value)`.

Validation performed: Every generated file is syntactically valid TypeScript. Step counts and precondition counts match B1 plans (verified via `b2-tests.json`). Generation rate: 257/257 = 100%.

Deterministic guarantee: Same `b1-plans.json` → byte-identical generated test files and `b2-tests.json`. Determinism verified by two independent runs.

6. Artifact Statistics

6.1 A1 Multigraph Statistics

Subject	Nodes	Edges	Structural	Executable	Modules	Routes	Components	Widgets	Services	External
posts-users-ui-ng	64	131	100	31	2	8	11	41	2	0
spring-petclinic-angular	191	422	294	128	16	24	22	120	9	0
heroes-angular	67	88	68	20	5	5	17	28	2	10
softscanner-cqa-frontend	37	74	57	17	3	1	10	21	2	0
ever-traduora	247	499	384	115	4	20	45	152	26	0
airbus-inventory	59	131	90	41	1	7	10	36	5	0
Total	**665**	**1345**	**993**	**352**	**31**	**65**	**115**	**398**	**46**	**10**

Multigraph integrity hashes (first 16 hex characters of SHA-256, for audit reproducibility):

Subject	multigraphHash (prefix)
posts-users-ui-ng	`4636b593477f2d46...`
spring-petclinic-angular	`375813f9f590284d...`
heroes-angular	`163aaa8d55057428...`
softscanner-cqa-frontend	`ed8d0e730d76b1fc...`

Subject	multigraphHash (prefix)
ever-traduora	`37af82c171824873...`
airbus-inventory	`d850a952b9b7d45d...`

6.2 A2 Workflow Statistics

Workflow counts and classification:

Subject	Total WFs	FEASIBLE	CONDITIONAL	PRUNED	Enum Routes	Trigger Edges
posts-users-ui-ng	18	12	6	0	7	18
spring-petclinic-angular	74	40	34	0	22	74
heroes-angular	19	19	0	0	4	19
softscanner-cqa-frontend	16	15	1	0	1	16
ever-traduora	109	46	63	0	18	109
airbus-inventory	21	13	8	0	6	21
Total	**257**	**145 (56%)**	**112 (44%)**	**0 (0%)**	**58**	**257**

CONDITIONAL reasons by subject (a single workflow may have multiple reasons):

Subject	Guards	Params	FormValid	UI Gates	Unresolved
posts-users-ui-ng	0	2	3	2	0
spring-petclinic-angular	0	19	12	9	0
heroes-angular	0	0	0	0	0
softscanner-cqa-frontend	0	0	1	0	0
ever-traduora	21	12	13	45	1
airbus-inventory	3	0	4	1	0

Trigger edge kind distribution:

Subject	WNR	WNE	WTH	WSF
posts-users-ui-ng	6	0	9	3
spring-petclinic-angular	8	0	54	12
heroes-angular	3	11	5	0
softscanner-cqa-frontend	0	0	15	1
ever-traduora	21	0	75	13
airbus-inventory	5	0	12	4
Total	**43 (17%)**	**11 (4%)**	**170 (66%)**	**33 (13%)**

WTH (WIDGET_TRIGGERS_HANDLER) workflows dominate at 66%, reflecting the prevalence of event handler bindings in Angular applications. WNE (external navigation) workflows are all found in heroes-angular (11 external link targets from the About page).

6.3 B1 Intent and Plan Statistics

Intent counts by verdict:

Subject	Total Intents	FEASIBLE	CONDITIONAL	PRUNED
posts-users-ui-ng	18	12	6	0
spring-petclinic-angular	74	40	34	0
heroes-angular	19	19	0	0
softscanner-cqa-frontend	16	15	1	0
ever-traduora	109	46	63	0
airbus-inventory	21	13	8	0
Total	**257**	**145**	**112**	**0**

Plan precondition types:

Subject	Total Plans	auth-setup	navigate-to-route	dialog-open	Skipped
posts-users-ui-ng	18	0	18	0	0
spring-petclinic-angular	74	0	74	0	0
heroes-angular	19	0	19	2	0
softscanner-cqa-frontend	16	0	16	0	0
ever-traduora	109	94	109	5	0
airbus-inventory	21	12	21	2	0
Total	**257**	**106 (41%)**	**257 (100%)**	**9 (4%)**	**0 (0%)**

106 of 257 plans (41%) require an `auth-setup` precondition — all from the two authenticated subjects (ever-traduora: 94, airbus-inventory: 12). Every plan includes a `navigate-to-route` precondition, ensuring that each test begins at a known URL state. 9 plans across 3 subjects (heroes-angular, ever-traduora, airbus-inventory) include a `trigger-dialog-open` precondition, reflecting the detection of dialog/modal component composition in A1.

Action step distributions:

Subject	Total Steps	avg/plan	click	clear-and-type	submit	select	wait-for
posts-users-ui-ng	41	2.3	15	21	3	0	2
spring-petclinic-angular	132	1.8	62	46	12	3	9
heroes-angular	19	1.0	19	0	0	0	0
softscanner-cqa-frontend	21	1.3	15	5	1	0	0
ever-traduora	180	1.7	96	24	13	2	45
airbus-inventory	35	1.7	17	10	4	3	1
Total	**428**	**1.7**	**224 (52%)**	**106 (25%)**	**33 (8%)**	**8 (2%)**	**57 (13%)**

The `wait-for-element` steps in ever-traduora (45 out of 180) reflect the high proportion of CONDITIONAL workflows with expression-gated UI — B1 emits explicit element-wait steps for widgets whose visibility or enabledness depends on runtime expressions.

Locator strategy distribution across all plans:

Strategy	Count	%	Example
tag-position	169	39.5%	`button:nth-of-type(2)`
name	45	10.5%	`[name="lastName"]`

Strategy	Count	%	Example
routerlink	68	15.9%	`[routerlink="/owners"]`
formcontrolname	60	14.0%	`[formcontrolname="email"]`
data-testid	23	5.4%	`[data-testid="owner-submit"]`
aria-label	18	4.2%	`[aria-label="Add Hero"]`
id	24	5.6%	`#firstName`
placeholder	1	0.2%	`[placeholder="Search"]`
other	20	4.7%	custom/href
Total	**428**		

`tag-position` is the most frequent fallback strategy (39.5%), indicating that many Angular template elements lack stable identifying attributes (`id` , `name` , `formcontrolname` , `aria-label`). `routerlink` and `formcontrolname` together account for ~30% of locators, reflecting good Angular-specific attribute extraction.

6.4 B2 Generated Test Statistics

Subject	Tests Generated	Skipped	Pre-conds	Steps	Post-conds	avg Steps/test
posts-users-ui-ng	18	0	18	41	18	2.3
spring-petclinic-angular	74	0	74	132	74	1.8
heroes-angular	19	0	21	19	19	1.0
softscanner-cqa-frontend	16	0	16	21	16	1.3
ever-traduora	109	0	208	180	109	1.7
airbus-inventory	21	0	35	35	21	1.7
Total	**257**	**0**	**372**	**428**	**257**	**1.7**

All 257 tests are generated with 0 skipped. Every test includes exactly one postcondition (always `assert-url-matches`). The 257 `.test.ts` files are located under `output/<subject>/tests/` with hash-based filenames (e.g., `17cc4bd6_AboutComponent_WNE.test.ts`).

6.5 Difficulty Class Distribution

GT entries are categorized by difficulty class (D1–D7) as defined in `approach.md` :

Class	Description	posts	petclinic	heroes	cqa	traduora	airbus	Total
D1	Simple FEASIBLE (click/nav, no constraints)	12	40	8	13	46	11	**130**
D2	Form submission (FormValid gate)	3	12	0	1	10	4	**30**
D3	Guard-protected navigation	0	0	0	0	7	3	**10**
D4	Parameterized route navigation	2	13	0	0	0	0	**15**
D5	Combined constraints (visibility, mixed)	1	9	0	0	46	1	**57**
D6	Dialog/modal interaction	0	0	0	2	0	2	**4**
D7	External URL navigation	0	0	11	0	0	0	**11**
Total		**18**	**74**	**19**	**16**	**109**	**21**	**257**

D1 (simple FEASIBLE) accounts for 130/257 (51%) of all workflows. D5 (combined constraints) is heavily concentrated in ever-traduora (46), which reflects the complex expression-gated UI in that application. D7 (external navigation) exists exclusively in heroes-angular (11 external link targets).

7. Coverage Metrics

7.1 Generation Coverage (C1 + C2) — Evaluated

Definition: Generation coverage measures the fraction of workflows in *W* for which the pipeline produces a valid ActionPlan (C1) and a syntactically valid test file (C2).

Formula:

- C1 = `|plans produced with skipped=false| / |W|`
- C2 = `|tests generated with skipped=false| / |W|`

Measurement: C1 and C2 are read directly from `b1-plans.json` (`.stats.totalPlanned` , `.stats.skipped`) and `b2-tests.json` (`.stats.generated` , `.stats.skipped`) artifacts.

Results:

Subject	W	Plans (C1)	C1 Rate	Tests (C2)	C2 Rate
posts-users-ui-ng	18	18	100.0%	18	100.0%
spring-petclinic-angular	74	74	100.0%	74	100.0%
heroes-angular	19	19	100.0%	19	100.0%
softscanner-cqa-frontend	16	16	100.0%	16	100.0%
ever-traduora	109	109	100.0%	109	100.0%
airbus-inventory	21	21	100.0%	21	100.0%
Total	**257**	**257**	**100.0%**	**257**	**100.0%**

Analysis: 100% generation coverage across all subjects demonstrates that the planning and code generation stages are complete and handle all workflow types including auth-protected, parameterized, form-submission, dialog-modal, and external navigation workflows. No workflow in *W* was skipped.

Important caveat: Generation coverage (C1+C2) is a necessary but not sufficient condition for execution coverage (C3). A generated test that is syntactically valid may still fail at execution time due to:

- Entity data not present in the running application's database
- Locator strategies failing to match actual rendered elements
- Timing issues requiring extended waits
- Application routing changes between extraction time and execution time

7.2 Execution Coverage (C3) — Future Stage

Definition: `|E| / (|W| - |W_app_not_ready|)` where:

- *E* = set of workflows for which at least one B3 test run produces `PASS`
- *W_app_not_ready* = workflows failing solely due to `FAIL_APP_NOT_READY` outcome (environment deficiency, not test failure)
- PRUNED workflows are excluded from *W* in the denominator

Why not yet evaluated: B3 (test execution with bounded retry) has not been implemented. Executing 257 tests across 6 subjects requires each subject application to be running and correctly seeded. The execution-readiness audit (`docs/analysis/phase-b/execution-readiness-audit.md`) documents the following data requirements:

Category	Workflows	Notes
No entity dependency	125 (49%)	Executable with current manifest alone
Require auth credentials	106 (41%)	Auth-setup preconditions; credentials in manifest
Require entity data in DB	129 (50%)	Route params reference specific entities
External navigation (no app)	11 (4%)	External URL links; execution may differ

Note: categories overlap (a workflow may require both auth and entity data).

Expected denominator: After excluding FAIL_APP_NOT_READY, the C3 denominator will be at most 257 (all workflows), subject to environment quality. If all 6 applications can be started with seed data, the denominator equals 257.

7.3 Validity Coverage (C4) — Future Stage

Definition: Oracle strength — the fraction of passed executions for which the postconditions constitute a meaningful semantic assertion beyond URL-match and no-crash.

Current postcondition set: Every generated test asserts:

1. `assert-url-matches` : the current URL contains the expected terminal route path (with route params substituted). This is a first-order functional check — it confirms that navigation occurred as expected.
2. `assert-no-crash` : the test reaching the postcondition implicitly asserts no exception was thrown during execution.

What is not yet asserted:

- No assertion on page content (elements present/absent on success)
- No assertion on response data displayed after form submission
- No assertion on API calls made or side effects produced
- No comparison against known baseline states

C4 is deferred until B3 execution infrastructure is stable and a richer postcondition vocabulary can be evaluated.

8. Determinism Verification

8.1 Motivation

Determinism is a necessary condition for reproducible evaluation. If any pipeline stage produces different outputs across runs, then:

- Coverage metrics cannot be reproducibly compared
- GT validation results cannot be trusted
- Differential analysis between subjects is unreliable

8.2 Determinism Protocol

Determinism is verified by running the pipeline twice on each subject and comparing the SHA-256 hash of the output artifact. Verification scripts are provided for each stage.

Stage	Script	Artifact Compared
A1 + A2	`npm run verify:determinism`	`a1-multigraph.json` + `a2-workflows.json`
B0	`npm run verify:b0-determinism`	`logs/b0-summary.json`
B1 intents	`npm run verify:b1-determinism`	`b1-intents.json` (all subjects)
B1 plans	`npm run verify:b1-plan-determinism`	`b1-plans.json` (all subjects)

Stage	Script	Artifact Compared
B2	`npm run verify:b2-determinism`	`b2-tests.json` + test files (all subjects)

8.3 Determinism Evidence

All verification scripts passed at the time of this evaluation:

Check	Status	Evidence
A1 multigraph	PASS	SHA-256 hashes match across 2 independent runs for all 6 subjects
A2 workflows	PASS	SHA-256 hashes match; workflow IDs are trigger edge IDs (deterministic)
B0 summary	PASS	Validation results are pure functions of manifest + A2 data
B1 intents	PASS	SHA-256 hashes match for all 6 `b1-intents.json` files
B1 plans	PASS	SHA-256 hashes match for all 6 `b1-plans.json` files
B2 test files	PASS	SHA-256 of `b2-tests.json` matches; test file content identical

8.4 Sources of Non-Determinism (Mitigated)

The following potential sources of non-determinism were identified and addressed:

Source	Mitigation
File traversal order	All file discovery uses `readdirSync` sorted by path; no OS-level ordering relied on
Map/Set iteration order	A1 and A2 use arrays with explicit sort; no unordered Set iteration in output paths
Node/edge ID generation	All IDs derived from file paths + class names + source offsets (deterministic)
External node ID	FNV-1a 32-bit hash of URL string (deterministic)
Route deduplication	Tie-breaking by component binding specificity, then lexicographic
Redirect edge selection	Deterministic: `(edge.to asc, edge.id asc)`
Effect edge ordering	CCS edges sorted by `callsiteOrdinal`; ties broken by `edge.id`
Workflow ordering	Sorted by `id` ascending in output
Filename generation (B2)	DJB2 hash of workflow ID → 8-char hex prefix

9. Comparison with AutoE2E

9.1 AutoE2E Overview

AutoE2E (Allan et al., "Feature-Driven End-To-End Test Generation," ICSE 2025) is an LLM-based approach for E2E test generation that operates through runtime exploration. Its pipeline:

1. Launches a Chrome WebDriver instance targeting the application at `base_url`
2. Extracts candidate actions from the live DOM (clicks, form interactions)
3. Executes up to 30 actions, tracking new DOM states by content hash
4. For each action leading to a new state: extracts a page description via screenshot + Claude Sonnet; infers features (LLM-generated strings) via Claude Sonnet + OpenAI embeddings; deduplicates features by vector similarity
5. Marks features as "final" (completed by an action) via Claude Haiku
6. Writes results to `report/{APP_NAME}.json` (state graph) and MongoDB (feature + action-feature mappings)

What AutoE2E does not do: AutoE2E does not generate executable test files. The repository title claims "End-To-End Test Generation" but the pipeline terminates after writing the state graph. This was confirmed by static code inspection (no test-framework imports, no `.spec` / `.test` / `.ts` output code) and by running the pipeline on all 6 subjects (confirmed by the companion AutoE2E benchmark evaluation report).

9.2 Conceptual Differences in Coverage Model

The fundamental methodological difference is in how the interaction space is constructed and how coverage is defined over it.

AutoE2E coverage model:

- Interaction space: the set of states and actions observed during a bounded crawl (30-action ceiling)
- Coverage: measured as features discovered / features attempted, or states reached / total actions
- Denominator: exploration-bounded; varies between runs and with different configurations
- Coverage reflects how well the exploration policy discovers behavior, not how completely it exercises the implementation

This work's coverage model:

- Interaction space: W = finite, classified workflow set derived statically from source code
- Coverage: `|E| / |W|` — fraction of workflows for which at least one test passes
- Denominator: fixed and implementation-grounded; identical across approaches evaluated over the same W
- Coverage reflects how well the realization strategy exercises the defined workflow space

This distinction is load-bearing: when two approaches are evaluated over the same W, coverage differences reflect differences in realization capability. When each approach defines its own denominator, observed differences may reflect denominator differences rather than behavioral differences.

9.3 Coverage Conceptual Mapping

AutoE2E's "features" and this work's "workflows" are not directly equivalent:

- A **feature** in AutoE2E is an LLM-inferred string description of an application capability (e.g., "add new character to collection"). Features are not grounded in the implementation — they may be hallucinated, duplicated, or over-specific.
- A **workflow** in this work is a concrete path through the A1 multigraph from a start route to a terminal node, with all constraints explicitly encoded. Workflows are grounded in source code and auditable.
- Both approaches' test cases are mapped to W by extracting the action sequence (navigate, click, type, submit) and matching it to an entry-to-terminal workflow path.

9.4 AutoE2E 6-Subject Benchmark Evidence

A companion benchmark evaluation (see `docs/validation/autoe2e-benchmark-evaluation-report.md`) executed AutoE2E on all six subjects used in this evaluation. Key findings:

Subject	States	Actions Executed	Features Stored	Binding Constraint
Spring PetClinic	9	30	46	30-action ceiling
Ever Traduora	3	13	5	Authentication wall
Posts & Users	7	30	~37	30-action ceiling
Heroes Angular	5	30	~25	30-action ceiling
Airbus Inventory	1	6	0	Auth wall + finality
SoftScanner CQA	9	30	6	30-action ceiling

Key structural limitations of AutoE2E across the six subjects:

1. **No authentication support:** AutoE2E has no credential management. Ever Traduora (all features require login) and Airbus Inventory (JWT-protected) are effectively blocked after 3 and 1 states respectively.

2. **Form submission PetClinic-specific:** Form submission requires `data-testid` / `data-formid` / `data-submitid` HTML attributes present only in the PetClinic benchmark. All other subjects have zero form submissions.
3. **30-action ceiling:** For applications with dense interaction surfaces (Posts & Users: 95 candidate actions, SoftScanner CQA: 452 candidate actions), the ceiling is reached before exploring all accessible states.
4. **No test file output:** Confirmed by static code inspection and all six runs — AutoE2E produces state graphs and feature metadata, not executable tests.

9.5 Side-by-Side Comparison Table

Dimension	This Work	AutoE2E
Interaction space construction	Static: Angular AST + constraint extraction	Dynamic: runtime crawl (30 actions)
Interaction space grounding	Implementation-grounded (source code)	Exploration-bounded (observed behavior)
Coverage denominator	Fixed |W| (same across approaches)	Not explicitly defined; exploration-dependent
Constraint handling	Explicit: guards, params, UI atoms in C(w)	Implicit: auth wall blocks navigation entirely
Test generation output	Executable TypeScript Selenium tests	Not produced (state graph + feature metadata only)
Test generation strategy	Deterministic from ActionPlans	Not applicable
Auth support	Manifest-driven credentials + authSetup	Not supported
Form interaction	Schema-aware from A1 extraction	Requires `data-testid` instrumentation
Parameterized routes	Manifest `routeParamValues` binding	Empty backend prevents dynamic route navigation
Determinism	Byte-identical across runs	LLM-dependent; non-deterministic
Angular specificity	Required (ts-morph + Angular compiler)	General web crawler; Angular incidental
Prior execution required	No (static analysis only)	Yes (running application required)
Subjects fully covered	6/6 (all workflows planned and generated)	1/6 meaningful coverage (PetClinic with seed data)
Execution coverage (6 subjects)	Pending (B3 not yet implemented)	Not measurable (no tests produced)

10. Experimental Observations

10.1 Strengths of the Approach

Complete coverage of the workflow space. The approach enumerates every enabled trigger edge in every active component context, guaranteeing that no workflow in *W* is silently dropped. The PRUNED workflow count is zero across all subjects — all 257 workflows are either FEASIBLE or CONDITIONAL, meaning they are considered potentially executable. This is by design: pruning requires a provable literal contradiction (e.g., `visibleLiteral=false`), not a mere heuristic.

Constraint-aware planning. The approach correctly handles authentication (106 plans with `auth-setup`), route parameter binding (all 74 PetClinic parameterized routes resolved), form schemas (33 WSF-triggered plans with field-level form data), and dialog opening (9 plans with `trigger-dialog-open`). These cases are systematically handled, not discovered by luck during exploration.

Determinism as a scientific property. Byte-identical outputs across independent runs mean that evaluation results are reproducible. Any two researchers running the pipeline on the same codebase will obtain identical results. This is not a property that exploration-based approaches can easily provide.

Source-grounded auditability. Every node, edge, and workflow step is backed by `SourceRef` (file + character offsets). When a generated test fails, the failure can be traced back to a specific source code location. This is essential for debugging locator failures and for maintaining the pipeline as the application evolves.

Separation of construction from realization. Phase A is run once per subject; Phase B can be re-run with different manifests (e.g., different credentials or route param values) without re-running A1/A2. This separation also means that workflow space changes are clearly attributable to code changes, not to changes in exploration policy.

10.2 Performance by Subject

posts-users-ui-ng: Straightforward extraction and planning. The flat module structure, standard reactive forms, and routerLink navigation are well within A1's modeling capabilities. 0 unresolved navigation targets. The 6 CONDITIONAL workflows reflect parameterized routes (`/posts/:id` , `/users/:id`) and form validity requirements.

spring-petclinic-angular: The most demanding subject for planning (74 workflows, 34 CONDITIONAL, 19 parameterized). The approach correctly resolves previously problematic navigation patterns: array navigation (`['/owners', id]` → `/owners/:id`), interpolation navigation (`/owners/{{owner.id}}` → `/owners/:id`), and route deduplication across lazy-loaded modules. All 74 tests are generated with correct route param bindings.

heroes-angular: The cleanest extraction result (19 FEASIBLE, 0 CONDITIONAL). Lazy-loaded module deduplication correctly collapses duplicate `/heroes` and `/villains` routes. The 11 external navigation workflows (WNE) correctly target external URLs. The approach correctly handles `@Injectable({ providedIn: 'root' })` for HeroService and VillainService.

softscanner-cqa-frontend: Angular 17 standalone components are handled transparently (the Angular compiler API abstracts over NgModule vs standalone). Dialog composition detection correctly identifies 2 dialog-related workflows that require `trigger-dialog-open` preconditions.

ever-traduora: The largest and most complex extraction (247 nodes, 499 edges, 109 workflows). The approach correctly models 4 module layers, 20 routes with ROUTE_HAS_CHILD hierarchy, 26 services, and 3 redirect edges. 94 auth-setup preconditions are correctly derived from `AuthGuard` / `CanGuard` on guarded routes. The 45 UI gate CONDITIONAL workflows reflect expression-gated buttons and links in the application's UI, which the approach correctly identifies as needing runtime evaluation.

airbus-inventory: Flat single-module architecture with JWT guards. The approach correctly emits `MODULE_PROVIDES_SERVICE` edges for all 5 services (all declared in the single AppModule), handles the root route redirect (`/` → `/dashboard`), and derives auth-setup preconditions for the 12 guarded product routes.

10.3 Structural Advantages over Feature-Based Approaches

Complete coverage of guarded routes. AutoE2E discovered 0 features behind JWT authentication in Airbus Inventory and only pre-auth features in Ever Traduora. This work generates test plans for all 21 Airbus workflows and all 109 Traduora workflows, including all 94 auth-protected ones, because auth handling is part of the planning substrate, not a runtime dependency.

Systematic form interaction. AutoE2E submitted 0 forms across 5 of 6 subjects (the exception requiring special HTML attributes). This work generates field-level form interaction for all 33 WSF-triggered workflows across 5 subjects, using A1-extracted `formControlName` , `name` , and `id` attributes.

Parameterized route coverage. AutoE2E could not reach dynamic routes (`/users/:id` , `/owners/:id/pets/:petId`) in subjects with empty or auth-blocked backends. This work generates correct route param substitutions for all 34 parameterized PetClinic workflows and 14 Posts & Users parameterized workflows using manifest-provided values.

10.4 Insights from Application Analysis

Several structural patterns emerged from the multi-subject analysis:

1. **Guard guard-satisfying accounts are essential for large applications.** Ever Traduora has 109 workflows; 94 (86%) require auth-setup. Without manifest-driven credential binding, 94 tests would be trivially failing from the start. The manifest's `guardSatisfies` mechanism correctly identifies which account to use for each workflow.
2. **Expression-gated UI is pervasive.** 57 of 257 workflows (22%) are CONDITIONAL due to expression-based UI gates (D5 difficulty class). These workflows cannot be statically proven feasible but are not provably infeasible either. The approach correctly classifies them as CONDITIONAL and includes them in the test generation, with `wait-for-element` steps to handle conditional rendering.
3. **Tag-position locators dominate due to missing stable IDs.** 39.5% of locators fall back to `tag-position` (`nth-of-type`). This is a direct consequence of Angular developers not adding `id` , `data-testid` , or `aria-label` attributes to interactive elements. This locator strategy is the weakest for execution stability — position-based selectors are fragile if the rendered order changes.

4. **Zero PRUNED workflows** across all 6 subjects indicates that the Angular framework does not typically produce statically provable contradictions in UI state. The classifier is conservative by design (no heuristic pruning), which is appropriate for a coverage-maximization goal.

11. Limitations

11.1 Execution Readiness

B3 not implemented. The most critical limitation of this evaluation is that generated tests have not been executed against live applications. Generation coverage (C1+C2 = 100%) establishes that the planning and code generation stages are correct, but it does not prove that tests will pass when run.

Entity data dependency. 129 of 257 workflows (50%) navigate to parameterized routes (e.g., `/owners/1`, `/projects/test-project-id`) that assume the corresponding entity exists in the application's data store. If the backend starts empty, tests for these workflows will fail with `FAIL_ELEMENT_NOT_FOUND` or `FAIL_ASSERTION`. Entity provisioning is the user's responsibility; the manifest's `seedDataNotes` field documents requirements but does not enforce them.

Authentication session management. 106 plans include `auth-setup` preconditions. The generated auth code navigates to the login route and submits credentials using manifest-specified selectors. If the authentication flow changes (e.g., CAPTCHA added, redirect behavior modified), these preconditions will fail silently or produce `FAIL_AUTH` outcomes.

11.2 Locator Stability

Tag-position locators are fragile. 39.5% of locators use `nth-of-type` selectors. These are correct at generation time (based on A1's template analysis) but may break if the application adds or reorders elements in the same component context. Stronger locators (`id`, `data-testid`, `aria-label`) are preferred but require application instrumentation.

Component ancestor scoping assumes stable selectors. Component-scoped locators assume that the component's CSS selector (e.g., `app-owner-detail`, `app-project-locales`) is stable and unique in the rendered DOM. If multiple instances of the same component are rendered simultaneously, selector uniqueness breaks.

11.3 Angular-Specific Assumptions

Framework scope is Angular only. The A1 extraction layer uses `ts-morph` for TypeScript AST analysis and `@angular/compiler` for template analysis. The approach does not generalize to React, Vue, Svelte, or other frontend frameworks. However, the approach could be extended to other frameworks by extending the parser and analyzer layers.

Angular version sensitivity. Extraction was validated on Angular 12–18. Angular 19+ changes to standalone components, signals, or the router API may require A1 updates. The approach is not framework-version-agnostic.

11.4 Workflow Space Completeness

Single-trigger model. Each workflow in *W* begins with exactly one trigger edge. Multi-trigger workflows (e.g., "add a pet owner, then add their pet") that require prior state setup beyond what preconditions can express are not modeled.

Known gaps from A1 extraction:

- `@Output()` event chains: triggers on `@Output()` directive events from child components are not traced through the event binding chain
- Ancestor `*ngIf` propagation: an `*ngIf` on a parent element that conditionally hides a trigger widget is not always detected
- `RouterService` indirection: service-mediated navigation (calling a helper service that wraps `Router.navigate`) may not be resolved if the indirection level exceeds bounded transitive following depth

11.5 Postcondition Oracle Strength

URL-match postconditions are necessary but not sufficient. The generated tests assert that the terminal URL contains the expected path. This does not verify:

- That form submission actually created, updated, or deleted a record
- That displayed data matches expected entity state
- That error conditions are handled correctly (e.g., 404 on missing entity)

A test may pass (correct terminal URL) while the underlying operation failed silently. This is the C4 limitation.

11.6 Ground Truth Validation Scope

GT validates planning decisions, not execution outcomes. The 257/257 GT match guarantees that the B1 planner produces the correct plan structure for each workflow. It does not guarantee that:

- The plan will execute successfully on the live application
- The locators will match rendered elements
- The route param values in the manifest are correct for the actual application state

12. Future Work

12.1 B3: Test Execution with Bounded Retry

The immediate next step is implementing B3 — executing the 257 generated tests against their respective applications with bounded retry logic:

- **Level 1:** Execute as-is. Record `PASS` or failure outcome.
- **Level 2:** Retry with increased implicit wait timeouts and explicit `wait-for-element` insertion.
- **Level 3:** LLM-assisted locator or plan repair — use Claude to suggest alternative selectors or adjusted action sequences, re-generate the affected test, and retry.

Maximum 3 attempts per workflow. Each attempt produces an `ExecutionResult` entry in `b3-results.json`.

The `FAIL_APP_NOT_READY` outcome requires pre-run readiness checking: HTTP GET to `executionConfig.readinessEndpoint` (defaulting to `baseUrl`) before executing any tests for a subject.

12.2 Execution Coverage Computation (B4)

After B3, B4 computes tiered coverage metrics over the fixed *W*:

- **C1**: Already measured = 257/257 = 100%
- **C2**: Already measured = 257/257 = 100%
- **C3**: To be computed from B3 `ExecutionResult` artifacts: `|PASS| / (|W| - |FAIL_APP_NOT_READY|)`
- **C4**: Deferred — requires richer postcondition vocabulary

12.3 Entity Data Provisioning Improvements

The current approach leaves entity provisioning to the user. Future improvements will include:

- Automated seed data generation based on requirements (e.g., creating an owner with `id=1` using the application's API before running tests)
- Integration with database fixtures at the subject manifest level

12.4 Locator Robustness

To reduce the proportion of fragile `tag-position` locators (currently 39.5%), future work could:

- Extend the A1 locator extraction to cover `data-cy`, `data-e2e`, and other common test-attribution attributes
- Implement locator healing in B3 Level 2 retry: when `tag-position` fails, attempt sibling attribute strategies
- Provide a static report of widgets without stable locator attributes to guide manual instrumentation

12.5 Multi-Step Workflow Composition

The current single-trigger model does not capture workflows that require prior state setup. Future extensions could:

- Define composite workflows as ordered sequences of task workflows, where the terminal state of one workflow serves as the start state of the next
- Use the A1 multigraph's service call structure to infer state-creating prerequisites (e.g., "create owner" must precede "add pet for owner")

12.6 Framework Generalization

Extending the approach to React and Vue would require:

- Replacing the Angular-specific `@angular/compiler` template parser with framework-specific AST analyzers
- Adapting the routing model to React Router / Vue Router conventions
- Handling JSX/TSX template syntax in the widget extraction layer

The A2, B0, B1, B2 stages are framework-agnostic once the A1 multigraph is available; only A1 requires framework-specific implementation.

Appendix A: Generated Test File Example

The following is a complete generated test from `output/heroes-angular/tests/9bd04822_AboutComponent_WNE.test.ts` (D7 — external URL navigation, minimal complexity):

```
import { Builder, By, until, WebDriver } from 'selenium-webdriver';
import chrome from 'selenium-webdriver/chrome';
import assert from 'assert';

const BASE_URL = 'http://localhost:4200';
const IMPLICIT_WAIT = 10000;
const NAVIGATION_WAIT = 15000;

async function runTest(): Promise<void> {
  const options = new chrome.Options();
  options.addArguments('--headless');
  options.addArguments('--no-sandbox');
  options.addArguments('--disable-dev-shm-usage');

  const driver: WebDriver = await new Builder()
    .forBrowser('chrome')
    .setChromeOptions(options)
    .build();

  await driver.manage().setTimeouts({ implicit: IMPLICIT_WAIT });

  try {
    // PreCondition: navigate-to-route
    await driver.get(BASE_URL + '/about');
    await driver.wait(until.elementLocated(By.css('body')), NAVIGATION_WAIT);

    // Step 1: Click a (WIDGET_NAVIGATES_EXTERNAL)
    {
      const el = await driver.findElement(By.css('app-about'))
                              .findElement(By.css('a:nth-of-type(1)'));
      await el.click();
    }

    // PostCondition: assert-url-matches
    {
      const currentUrl = await driver.getCurrentUrl();
      assert.ok(
        currentUrl.includes('https://aka.ms/jp-free'),
        `Expected URL to contain https://aka.ms/jp-free, got ${currentUrl}`
      );
    }

    console.log('Test PASSED: <workflowId>');
  } finally {
    await driver.quit();
  }
}

runTest().catch((err) => {
  console.error('Test FAILED:', err);
  process.exit(1);
});
```

Appendix B: Generated Test File Example — Auth + Navigation

The following is an excerpt from `output/ever-traduora/tests/0050744a_ProjectLocalesComponent_WNR.test.ts` (D3+D4 — guard-protected navigation to parameterized route):

```
try {
  // PreCondition: auth-setup
  await driver.get(BASE_URL + '/login');
  {
    const usernameEl = await driver.findElement(
      By.css("input[formcontrolname='email']")
    );
    await usernameEl.clear();
    await usernameEl.sendKeys('admin@example.com');
    const passwordEl = await driver.findElement(
      By.css("input[formcontrolname='password']")
    );
    await passwordEl.clear();
    await passwordEl.sendKeys('admin123');
    const submitEl = await driver.findElement(
      By.css("button[type='submit']")
    );
    await submitEl.click();
    await driver.wait(until.urlContains('/'), NAVIGATION_WAIT);
  }

  // PreCondition: navigate-to-route
  await driver.get(BASE_URL + '/projects/test-project-id');
  await driver.wait(until.elementLocated(By.css('body')), NAVIGATION_WAIT);

  // Step 1: Wait for trigger to become visible (project$ | async)
  { /* ... wait-for-element for expression-gated link ... */ }

  // Step 2: Click div (WIDGET_NAVIGATES_ROUTE)
  {
    const el = await driver.findElement(By.css('app-project-locales'))
                           .findElement(By.css('[routerlink="[locale.locale.code]"]'));
    await el.click();
  }

  // PostCondition: assert-url-matches
  await driver.wait(async () => {
    const url = await driver.getCurrentUrl();
    return url.includes('/projects/test-project-id/translations/en');
  }, NAVIGATION_WAIT, 'Expected URL to contain /projects/test-project-id/translations/en');
}
```

Appendix C: Pipeline Artifact Locations

Artifact	Location
A1 multigraphs	`output/<subject>/json/a1-multigraph.json`
A2 workflow sets	`output/<subject>/json/a2-workflows.json`
B1 intent sets	`output/<subject>/json/b1-intents.json`
B1 plan sets	`output/<subject>/json/b1-plans.json`
B2 test files	`output/<subject>/tests/*.test.ts`
B2 metadata	`output/<subject>/json/b2-tests.json`
Subject manifests	`subjects/<subject>/subject-manifest.json`

Artifact	Location
Ground truth	`docs/analysis/phase-b/gt/<subject>.json`
AutoE2E report	`docs/validation/autoe2e-benchmark-evaluation-report.md`
B1 closure report	`docs/analysis/phase-b/b1-closure-report.md`
B2 closure report	`docs/analysis/phase-b/b2-closure-report.md`
Execution readiness audit	`docs/analysis/phase-b/execution-readiness-audit.md`